UNIR

Actividad 1 Desarrollo de un gestor de tareas con IA

Asignatura

Generalización de Código de Automatización en Desarrollo de Software con IA

Datos del alumno

Juan Pablo Arcos Merchan

Fecha

03/11/2026

Desarrollo de un gestor de tareas de estudio con IA

Estudio es una aplicación web local que reúne tareas de diferentes asignaturas, permite planificar entregas y conserva los datos en SQLite. El prototipo incorpora el filtro de tareas fuera de plazo solicitado tras revisar la primera versión y ha superado 42 métodos de prueba automatizados.

Necesidad y alcance

El usuario necesita consultar en un único lugar qué debe hacer, qué está realizando y qué ha terminado. La solución se dirige a una persona que trabaja en su propio equipo. No requiere registro, servicios externos ni conexión a internet para su uso habitual.

Requisitos funcionales

ID	Función	Aceptación
RF01	Crear tareas	Título obligatorio y campos opcionales validados
RF02	Consultar y persistir	Conservar tareas después de reiniciar
RF03	Editar	Modificar los seis campos de una tarea
RF04	Cambiar estado	Pendiente, en curso, completada y reapertura
RF05	Eliminar	Confirmación y posibilidad de cancelar
RF06	Buscar	Título, asignatura y notas sin distinguir mayúsculas
RF07	Filtrar	Estado y prioridad combinados con búsqueda
RF08	Resumen global	Contadores independientes de los filtros
RF09	Fuera de plazo	Fecha anterior a hoy y tarea sin completar

RF01 a RF08 corresponden al alcance inicial. RF09 se añadió mediante una petición expresa del usuario después del prototipo. La especificación está conservada en documentacion/proyecto_inicial.md.

Diseño técnico y decisiones

La interfaz HTML, CSS y JavaScript se comunica mediante JSON con un servidor HTTP escrito en Python. El servidor valida los datos y utiliza un repositorio SQLite. Todo el código de aplicación y pruebas fue generado por el asistente de IA dentro de esta sesión.

Flujo de datos

```
Navegador -> API HTTP local -> validación -> SQLite
Navegador <- respuesta JSON <- repositorio
```

Separación de responsabilidades

app.py contiene validate, TaskStore y Handler. La validación se puede comprobar sin abrir el navegador. TaskStore concentra las consultas parametrizadas. Handler expone rutas conocidas y sirve únicamente los tres recursos estáticos de la interfaz.

Operación	Ruta	Resultado normal
GET	/api/tasks	200 con listado y filtros de consulta
POST	/api/tasks	201 con la tarea creada
GET	/api/tasks/{id}	200 con la tarea o 404 si no existe
PUT	/api/tasks/{id}	200 con sustitución validada de campos
DELETE	/api/tasks/{id}	204 tras el borrado o 404

Modelo de datos

La tabla tasks almacena id, title, subject, notes, due_date, priority, status y created_at. La fecha límite se guarda como AAAA-MM-DD o texto vacío; created_at se registra en UTC. El identificador y la fecha de creación se conservan al editar.

Criterios técnicos

Se eligió SQLite para conservar datos sin instalar un servidor de base de datos. La biblioteca estándar reduce dependencias y facilita ejecutar el ejercicio. El cierre explícito de conexiones evita bloqueos de archivos en Windows. La interfaz inserta datos con textContent y el servidor limita tamaños, valida fechas y restringe orígenes de escritura.

El título admite 120 caracteres, la asignatura 80 y las notas 2000. La interfaz usa etiquetas, foco visible y diálogos nativos; el diseño móvil se comprobó a 390 píxeles de ancho.

Aplicación funcionando

El tablero permite revisar las tareas y acceder a las acciones principales. La captura se obtuvo de una ejecución real con Microsoft Edge y una base temporal de cuatro tareas de ejemplo.

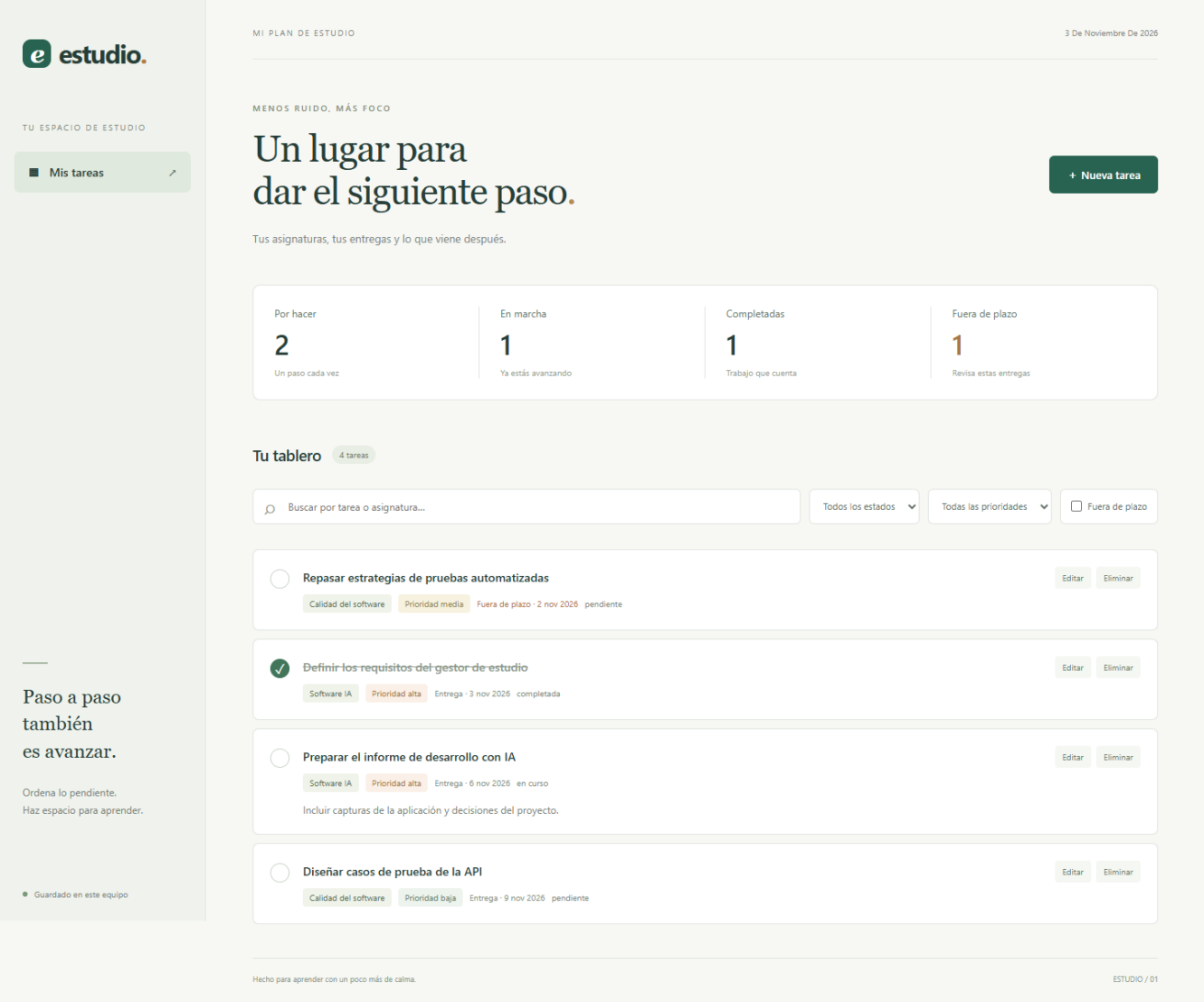


Figura 1. Vista completa de escritorio. Archivo original: evidencias/01_tablero.png. Los datos de ejemplo se usan solo para las evidencias.

Los contadores distinguen tareas pendientes, en curso, completadas y fuera de plazo. La búsqueda y los filtros reducen la lista visible, mientras los contadores siguen mostrando el total. La tarea completada permanece identificable y puede reabrirse.

La base de uso habitual se crea vacía en data/estudio.db. No contiene las tareas de las capturas.

Edición y mejora solicitada

El mismo formulario permite crear y editar tareas. El estado en curso se establece desde el selector; el botón circular del tablero completa o reabre la tarea.

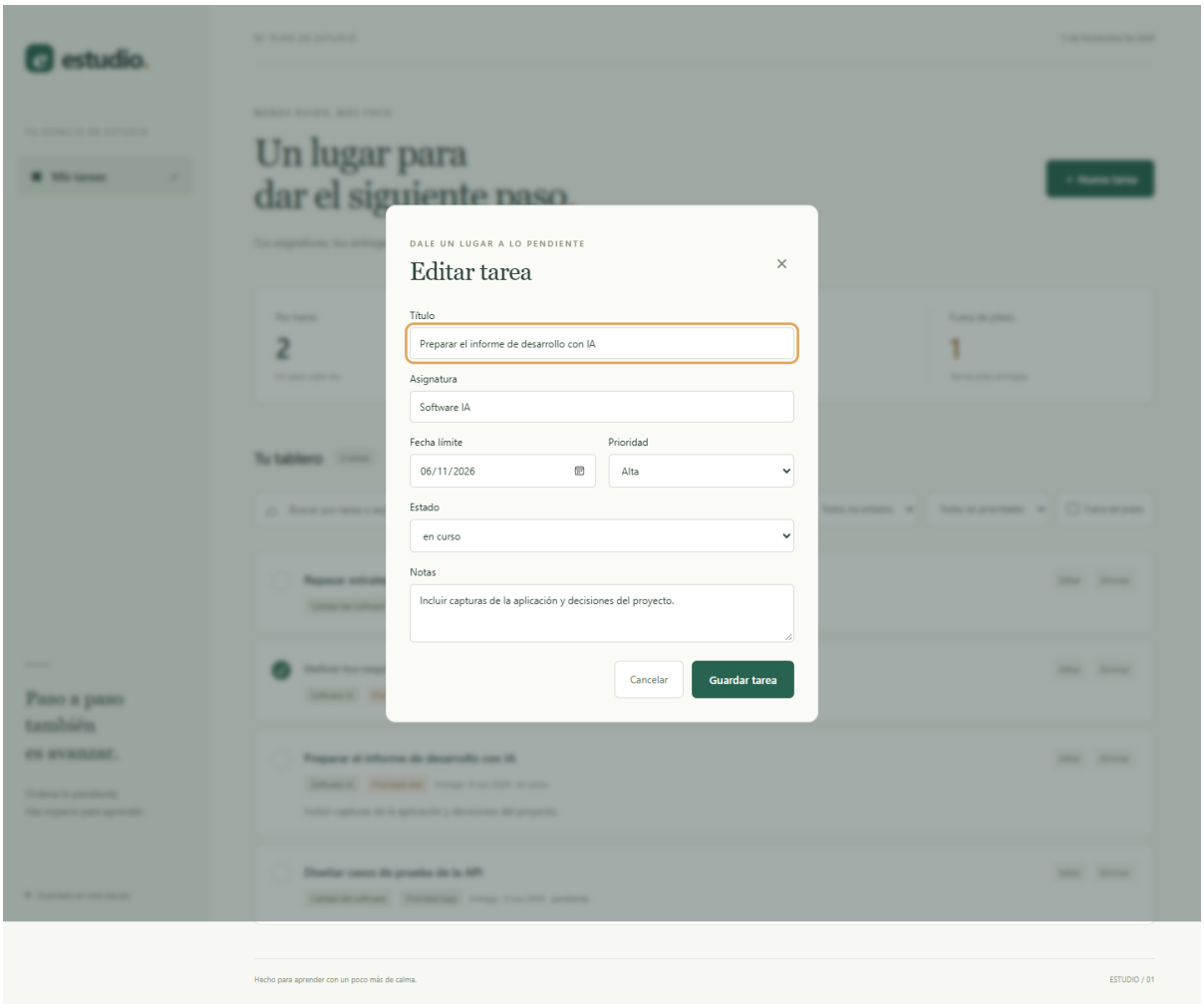


Figura 2. Formulario de edición con sus valores cargados. Archivo: evidencias/02_edicion.png.

Revisión del usuario

Petición literal: **«Añadir un filtro de tareas fuera de plazo»**. Se incorporó una casilla que combina este criterio con el resto de filtros. Una fecha anterior al día local del navegador identifica una tarea vencida solo cuando no está completada. Las tareas de hoy y las que no tienen fecha quedan excluidas.

La petición se convirtió en RF09 y en una prueba de interfaz con seis tareas que cubre ayer, hoy, mañana, ausencia de fecha, tareas completadas y combinación de filtros.

Interacción con la IA e iteraciones

Se conservan extractos reales de esta sesión y se distingue la participación del usuario de las decisiones del asistente. El historial resumido se incluye en documentacion/interacciones.md.

Concreción del proyecto

Usuario: «ayudame a realizar las actividades». Después de que el asistente leyera los enunciados y preguntara por el proyecto, el usuario eligió «Gestor de tareas de estudio con base de datos local (recomendado)».

Asistente: «Prepararé un gestor de tareas de estudio con asignaturas, fechas límite, prioridades, búsqueda y estados de progreso. Usará SQLite y se ejecutará en local».

Resultado: se fijaron necesidad, campos y ejecución local. La formulación técnica detallada y la elección de las herramientas de prueba fueron propuestas por el asistente.

Corrección guiada por resultados

La primera ejecución de 33 pruebas produjo 12 errores asociados a la limpieza de archivos SQLite en Windows. El contexto de transacción no cerraba las conexiones. El asistente incorporó un gestor de contexto con cierre en finally y la siguiente ejecución terminó sin errores. El log inicial se conserva para verificar el cambio.

Asistente: «La primera ejecución ha detectado un fallo real: las conexiones de SQLite quedaban abiertas y Windows impedía borrar las bases de datos temporales».

Nueva instrucción y verificación

Usuario: «Añadir un filtro de tareas fuera de plazo». **Asistente:** «Mostrará las tareas sin completar cuya fecha límite sea anterior a hoy y podrá combinarse con la búsqueda, el estado y la prioridad».

La ampliación se implementó en la interfaz y se comprobó con un nuevo caso de navegador. La batería de interfaz pasó de 8 a 9 métodos.

Análisis de la comunicación

La petición inicial necesitó concreción. La instrucción de mejora fue más eficaz porque identificó una función observable. La dirección técnica recayó ampliamente en el asistente; una mejora del proceso sería indicar desde el inicio criterios de aceptación y pedir explícitamente las pruebas de cada regla. No se atribuyen al usuario prompts técnicos que no escribió.

Resultados y entrega reproducible

El producto final permite completar el ciclo de gestión de una tarea y conservar los datos entre ejecuciones. Las pruebas comprobaron funciones de dominio, persistencia, API y flujos reales del navegador.

Nivel	Métodos	Resultado
Validación	8	Correctos
Repositorio SQLite	12	Correctos
API HTTP	13	Correctos
Interfaz Edge	9	Correctos
Total	42	Sin fallos ni errores

Ejecución local

```
cd estudio
python app.py
# Abrir http://127.0.0.1:8000
```

Se requiere Python 3.11 o posterior. requirements.txt indica que no hay paquetes externos de ejecución. En Windows también se incluye iniciar.cmd. El README explica puertos alternativos, copias de la base y ejecución de pruebas.

Limitaciones y valoración crítica

La ejecución correcta de 42 métodos no demuestra ausencia de defectos ni cobertura total del código. No hay autenticación, sincronización, avisos ni cifrado de la base. No se evaluaron carga, múltiples navegadores, lectores de pantalla ni despliegue público. El servidor escucha solo en 127.0.0.1 y el alcance es personal y local.

La IA permitió producir aplicación, pruebas y documentación, pero también generó inicialmente un defecto de gestión de recursos. Ejecutar las pruebas y conservar el error hizo posible corregirlo con evidencia. La revisión humana aportó una mejora funcional concreta; la siguiente iteración debería reforzar la especificación inicial y evaluar la usabilidad con tareas reales.

Material entregado

Código fuente, instrucciones, dependencias, requisitos, extractos de interacción, pruebas y evidencias se incluyen en el ZIP de entrega. La actividad 2 desarrolla el plan de pruebas y sus resultados. Fuentes de trabajo: enunciados psuiadesoft02_act1.docx y psuiadesoft02_act2.docx; código y logs del proyecto Estudio; extractos reales del chat del 3 de noviembre de 2026.